



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



MULTITHREADING IN JAVA

WAYS TO CREATE THREADS!

What is Thread

- Separate flow of execution in program is called Thread
- For every thread there is an independent job to do

Example:

- For 2 independent jobs we required 2 threads
- Both threads execute in same time so that overall time of execution will be reduced

Real Time example: Home Construction

- If 25 people work simultaneously it takes only 3 months
- If 5 people work for task it takes 1 Year

Different ways of creating Threads

We can define threads in 2 ways

- By extending Thread Class
- By implementing Runnable interface

Create Threads By extending Thread Class

- Create class that extends Thread Class
- Override run () method to define job of thread

```
Class Thread1 extends Thread
{
    @Override
    public run()
    {
        Define Job of thread1
    }
}

Class Thread2 extends Thread
{
    @Override
    public run()
    {
        Define Job of thread2
    }
}
```

By Default, every java program contains one thread called main thread

Job of Main Thread:

- JVM is responsible to start Main thread
- Main thread will execute main method to create child threads and it will execute child thread jobs
- Code inside run () method is job of the thread

Example:

```
public class MainThread
{
    public static void main(String args[])
    {
        Thread1 thread1 = new Thread1(); // Executed by main thread and creates child thread object
        thread1.start(); // Executed by main thread and starts child thread

        // After thread1.start we have 2 threads: Main and thread1

        for(int i=1; i<=10; i++) // Executed by main thread
        {
            System.out.println("Main Thread Running at: " + i);
        }
    }
}
```



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



MULTITHREADING IN JAVA WAYS TO CREATE THREADS!

Example Code: By Extending Thread class

```
class Thread1 extends Thread
{
    public void run()
    {
        for(int i=1;i<=10;i++)
        {
            System.out.println("Thread1 Running at: "+ i);
        }
    }
}

public class MainThread
{
    public static void main(String args[])
    {
        Thread1 thread1 = new Thread1();
        thread1.start();
        for(int i=1;i<=10;i++)
        {
            System.out.println("Main Thread Running at: "+ i);
        }
    }
}
```

Output:

```
<terminated> MainThread [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (Jun 7, 2022, 12:23:29 PM)
Main Thread Running at: 1
Main Thread Running at: 2
Main Thread Running at: 3
Main Thread Running at: 4
Main Thread Running at: 5
Main Thread Running at: 6
Main Thread Running at: 7
Thread1 Running at: 1
Main Thread Running at: 8
Thread1 Running at: 2
Main Thread Running at: 9
Thread1 Running at: 3
Main Thread Running at: 10
Thread1 Running at: 4
Thread1 Running at: 5
Thread1 Running at: 6
Thread1 Running at: 7
Thread1 Running at: 8
Thread1 Running at: 9
Thread1 Running at: 10
```

- In multithreading Program output sequence is not guarantee
- Output is based on Thread scheduler and its algorithm

Case 1: Thread Scheduler

- Thread Scheduler is a part of JVM
- Thread Scheduler is responsible for schedule thread execution
- If multiple threads waiting to get chance to executing its job TS is responsible to decide order of execution
- Order of execution is depending on Thread scheduler algorithm
- In multithreading environment, we can't predict the Exact output, but we can predict some possible outputs
- Thread scheduler algorithm varying from JVM to JVM
 - May one JVM use First come first serve
 - Round robin
 - Shortest job first



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



MULTITHREADING IN JAVA WAYS TO CREATE THREADS!

Defining Thread by Implementing Runnable Interface

- Way 1: Extending Thread class
 - Internally it implements Runnable interface
- Way 2: Define Thread BY implementing Runnable Interface
 - Runnable Interface present in Java.Lang Package and it contains only one method run () method
 - Runnable interface is an example for SAM interface

Internally working in both approach:

```
#Approach 1

MyThread --- Thread Class ----- Runnable(r)

#Approach 2

MyRunnable --- Runnable(r)
```

In both the approach internally its implements runnable method

Example Program:

```
public class MyRunnable implements Runnable
{
    public void run()
    {
        for (int i = 1; i <= 10; i++)
        {
            System.out.println("Child Running at: " + i);
        }
    }
}

public class MyThread
{
    public static void main(String[] args)
    {
        MyRunnable runnableThread = new MyRunnable();
        Thread thread = new Thread(runnableThread); // Pass Target Runnable
        thread.start();
        for (int i = 1; i <= 10; i++)
        {
            System.out.println("Main Thread Running at: " + i);
        }
    }
}
```

OUTPUT:

```
<terminated> MyThread [Java Application] C:\Program Files\Java\jre1.8.0_201\bin\javaw.exe (Jun 7, 2022, 10:20:30 PM)
Main Thread Running at: 1
Child Running at: 1
Main Thread Running at: 2
Child Running at: 2
Main Thread Running at: 3
Child Running at: 3
Main Thread Running at: 4
Child Running at: 4
Main Thread Running at: 5
Child Running at: 5
```



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!

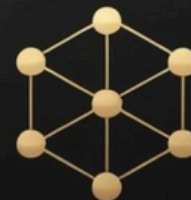


MULTITHREADING IN JAVA WAYS TO CREATE THREADS!

JAVA MULTITHREADING



EXTENDING THE
THREAD CLASS



IMPLEMENTING THE
RUNNABLE INTERFACE

Which one is best by extending Thread class or Implementing Runnable Interface?

- Implementing Runnable Interface way is BEST

Why Implementing Runnable Interface way is BEST?

Way 1: Extending Thread class

Example:

- In first approach we define thread by extending Thread class
- There is no chance to extending any other class
- Hence, we are missing Multiple inheritance benefit in First approach
- We can't extend More than one class

```
class Thread1 extends Thread, Thread2
{
    public void start()
    {
        //It will call actual start to create new thread and new thread will execute run() method
        super.start();
        System.out.println("Overriding start method and we won't get multithreading usage");
    }
    public void run()
    {
        for(int i=1; i<=10; i++)
        {
            System.out.println("Thread1 Running at: " + i);
        }
    }
}
```

Way 2: We can achieve Multiple inheritance

```
public class MyRunnable extends Thread1 implements Runnable
{
    public void run()
    {
        for (int i = 1; i <= 10; i++)
        {
            System.out.println("Child Running at: " + i);
        }
    }
}
```

- In 2nd approach we are defining thread by Implementing Runnable Interface
- And also, we can extend other class
- So, we can achieve Multiple inheritance also



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding on each topic.**
- **Once the interview series is complete, we will cover this topic in detail.**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com